🔍 Platform Vision vs. Codebase Implementation Audit

> [!NOTE] **Date:** February 2026
> **Scope:** Core backend architecture, RAG services, database schema, and tutor routing.
> **Objective:** A brutally honest technical audit mapping the declared platform vision against actual programmatic implementation within the backend codebase. Focuses on structural integrity, not UI polish.

🏗 Executive Summary

The audit reveals **significant architectural gaps** between the declared vision and the current codebase. The system currently implements standard Generative AI/RAG patterns (chunking + semantic search + prompt-based generation) rather than the strict, deterministic "Knowledge Graph" and "Interpreter" behavior prescribed by the vision. Active Human-in-the-Loop and Tutor Control systems are either conceptually drafted but not built, or they rely on manual, proactive pulling rather than programmatic enforcement and push alerts.

1️⃣ Knowledge Graph Discipline

Declared Vision: Knowledge Graph–based navigation (tree structure with static answer nodes), strict node isolation.

Data Point	Details
Status	🟡 Conceptual but not strictly present in code.
Locations	`schema.prisma` (`CourseChunk`, `TopicPromptSuggestion`), `ragService.ts`
Risk Level	> [!WARNING] **High** (Fundamental architectural mismatch)

🔍 Gap Description

The system is **not** a true Knowledge Graph. It utilizes a standard unstructured RAG approach. The database stores `course_chunks` with isolated `embedding` vectors. While there is a tree-like structure for *suggested prompts* (`parentSuggestionId` in `TopicPromptSuggestion`), the actual *knowledge* is flat, chunked text queried via Cosine Similarity (`1 - (embedding <=> ${Prisma.raw(vectorLiteral)})`). It does not traverse logical graph edges to find "static answer nodes".

💡 Recommended Action

Redesign the schema to model explicit `Nodes` and `Edges` with predefined static `Answers`, shifting from Semantic Search to Graph Traversal.

---

2 Guardrail Enforcement

Declared Vision: Strict guardrails, no hallucination, no external retrieval, strict out-of-graph query routing.

Data Point	Details
Status	🟡 Implemented but not enforced strictly.
Locations	`openAiClient.ts` (`generateAnswerFromContext`), `ragService.ts` (`buildPrompt`)
Risk Level	> [!CAUTION] **Medium**

🔍 Gap Description

External retrieval *is* successfully disabled (no agents/functions calling the web). The system reliably catches completely out-of-bounds queries via distance thresholds (returning 0 contexts triggers a fallback message). **However**, guardrails rely entirely on *Prompt Engineering* (`"Answer using only the provided contexts..."`). There is no deterministic programmatic validation of the LLM output against the source text.

💡 Recommended Action

Implement a programmatic output verification step (e.g., deterministic entailment checking) before sending the AI response to the user.

---

3 Digital Twin Behavior

Declared Vision: Interpreter of expert-approved content. Embeddings used *only* for matching, not generating.

Data Point	Details
Status	🟡 Partially implemented.
Locations	`openAiClient.ts` (`runChatCompletion`)
Risk Level	> [!CAUTION] **Medium**

🔍 Gap Description

Embeddings are used for matching context perfectly. However, the system fundamentally uses a Generative LLM (`client.chat.completions.create`) to build the final response. It is not acting as a pure "Interpreter" of static answers; it is dynamically generating novel sentences based on chunked context.

💡 Recommended Action

If strict "matching only" is required, the LLM should only be used to route the user's intent to a predefined static string ID, bypassing text generation entirely.

---

4 Training Wheels Logic

Declared Vision: Reduction of prompts across modules programmatically. Progression-based AI scaffolding logic.

Data Point	Details
Status	🟡 Implemented but static / not progressive.
Locations	`promptUsageService.ts`, `assistantService.ts`
Risk Level	> [!TIP] **Low**

🔍 Gap Description

A hard limit of 5 typed prompts per module is successfully enforced (`PROMPT_LIMIT_PER_MODULE = 5`). However, there is zero logic in the code that scales or reduces this number as the learner progresses (e.g., 10 prompts in Module 1, reducing to 2 in Module 5). It is a flat, static rate-limit rather than progression-based pedagogical scaffolding.

💡 Recommended Action

Make `PROMPT_LIMIT_PER_MODULE` dynamic based on the `moduleNo` or user's overall progress score.

---

5 Course Experience Layer

Declared Vision: Distinct listening, comprehension, and articulation components. Abstraction-heavy quizzes.

Data Point	Details
Status	🟡 Partially implemented.
Locations	`quizService.ts`, `schema.prisma` (`AssessmentQuestion`, `TopicProgress`)
Risk Level	> [!WARNING] **High**

🔍 Gap Description

The codebase currently supports standard Multiple Choice Questions (`mcq_options`) and text questions with strict percentage thresholds (`PASSING_PERCENT_THRESHOLD = 70`). There is no programmatic distinction in the database or service layers between "listening", "comprehension", and "articulation" phases. Everything maps to standard video/text content and MCQ quizzes.

💡 Recommended Action

Expand models to support abstraction queries (e.g., code execution blocks, short-answer deterministic grading) and explicitly separate topic phases.

---

6 Human-in-the-Loop Feedback

Declared Vision: Feedback loop fully closed, synchronous/asynchronous alerts, contextual thread re-entry.

Data Point	Details
Status	Partially implemented.
Locations	`activityEventService.ts`, `coldCall.ts`
Risk Level	> [!CAUTION] **Medium**

Gap Description

The data capture layer is excellent. The system accurately classifies granular events into `content_friction` and `attention_drift`. Cold calling (asynchronous threading) is built and works well. **The gap is the closed loop**. There is no push-alert system (Webhooks, WebSockets, or Email integrations) notifying Tutors of friction. Tutors must proactively pull data from `/me/courses/:courseId/progress`. Thread re-entry is not structurally enforced.

Recommended Action

Implement an Event Emitter or background job that monitors `activityEventService` and pushes real-time WebSocket alerts to the `tutor-backend`.

---

7 Tutor Control Layer

Declared Vision: Tutors can define tree structures, approve nodes, manage out-of-graph queries.

Data Point	Details
Status	Conceptual but not present in code.
Locations	`tutors.ts`
Risk Level	> [!WARNING] **High** (Critical missing functionality)

Gap Description

The Tutor API currently consists solely of read-only dashboards (`/enrollments`, `/progress`) and an AI analytics copilot (`/assistant/query`). There are **zero endpoints** in the codebase allowing a Tutor to intercept an out-of-graph query, edit a `CourseChunk`, or approve a `TopicPromptSuggestion`.

Recommended Action

Build dedicated CRUD routes for Content Management, specifically allowing Tutors to review failed RAG queries and intelligently inject manual answers into the graph.